

```
In [35]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
In [36]: df=pd.read_excel('Data Cleaning 12.08.26.xlsx')
df
```

Out[36]:

	name	planet_status	mass	mass_error_min	mass_error_max	mass_sini	mass_sini_error_min	mass_sini_error_max	radius	radius_err
0	109 Psc b	Confirmed	5.743	0.28900	1.01100	6.38300	0.07800	0.07800	1.152	
1	112 Psc b	Confirmed	NaN	0.00500	0.00400	0.03300	0.00500	0.00400	NaN	
2	112 Psc c	Confirmed	9.866	1.78100	3.19000	NaN	NaN	NaN	NaN	
3	11 UMi b	Confirmed	NaN	1.10000	1.10000	11.08730	1.10000	1.10000	NaN	
4	14 And Ab	Confirmed	NaN	0.23000	0.23000	4.68400	0.23000	0.23000	NaN	
...	...	...	...	...	...	...	...	...	...	
5981	YBP 401 b	Confirmed	NaN	0.05000	0.05000	0.46000	0.05000	0.05000	NaN	
5982	YSES 2 b	Confirmed	6.300	0.90000	1.60000	NaN	NaN	NaN	NaN	
5983	YZ Cet b	Confirmed	NaN	0.00028	0.00028	0.00220	0.00028	0.00028	NaN	
5984	YZ Cet c	Confirmed	NaN	0.00035	0.00035	0.00359	0.00035	0.00035	NaN	
5985	YZ Cet d	Confirmed	NaN	0.00038	0.00038	0.00343	0.00038	0.00038	NaN	

5986 rows × 10 columns



```
In [3]: df.head()
```

Out[3]:

	name	planet_status	mass	mass_error_min	mass_error_max	mass_sini	mass_sini_error_min	mass_sini_error_max	radius	radius_error_r
0	109 Psc b	Confirmed	5.743	0.289	1.011	6.3830	0.078	0.078	1.152	N
1	112 Psc b	Confirmed	NaN	0.005	0.004	0.0330	0.005	0.004	NaN	N
2	112 Psc c	Confirmed	9.866	1.781	3.190	NaN	NaN	NaN	NaN	N
3	11 UMi b	Confirmed	NaN	1.100	1.100	11.0873	1.100	1.100	NaN	N
4	14 And Ab	Confirmed	NaN	0.230	0.230	4.6840	0.230	0.230	NaN	N

In [4]: `df.tail()`

Out[4]:

	name	planet_status	mass	mass_error_min	mass_error_max	mass_sini	mass_sini_error_min	mass_sini_error_max	radius	radius_err
5981	YBP 401 b	Confirmed	NaN	0.05000	0.05000	0.46000	0.05000	0.05000	NaN	
5982	YSES 2 b	Confirmed	6.3	0.90000	1.60000	NaN	NaN	NaN	NaN	
5983	YZ Cet b	Confirmed	NaN	0.00028	0.00028	0.00220	0.00028	0.00028	NaN	
5984	YZ Cet c	Confirmed	NaN	0.00035	0.00035	0.00359	0.00035	0.00035	NaN	
5985	YZ Cet d	Confirmed	NaN	0.00038	0.00038	0.00343	0.00038	0.00038	NaN	

```
In [5]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 5986 entries, 0 to 5985  
Data columns (total 10 columns):  
#   Column                Non-Null Count  Dtype    
---  -  
0   name                   5986 non-null   object   
1   planet_status          5986 non-null   object   
2   mass                   2079 non-null   float64  
3   mass_error_min         2817 non-null   float64  
4   mass_error_max         2817 non-null   float64  
5   mass_sini              1202 non-null   float64  
6   mass_sini_error_min    1039 non-null   float64  
7   mass_sini_error_max    1039 non-null   float64  
8   radius                 4568 non-null   float64  
9   radius_error_min       4093 non-null   float64  
dtypes: float64(8), object(2)  
memory usage: 467.8+ KB
```

```
In [6]: df.shape
```

```
Out[6]: (5986, 10)
```

```
In [37]: df.isnull()
```

Out[37]:

	name	planet_status	mass	mass_error_min	mass_error_max	mass_sini	mass_sini_error_min	mass_sini_error_max	radius	radius_err
0	False	False	False	False	False	False	False	False	False	
1	False	False	True	False	False	False	False	False	False	True
2	False	False	False	False	False	True	True	True	True	True
3	False	False	True	False	False	False	False	False	False	True
4	False	False	True	False	False	False	False	False	False	True
...	...	...	...	...	...	...	...	...	...	...
5981	False	False	True	False	False	False	False	False	False	True
5982	False	False	False	False	False	True	True	True	True	True
5983	False	False	True	False	False	False	False	False	False	True
5984	False	False	True	False	False	False	False	False	False	True
5985	False	False	True	False	False	False	False	False	False	True

5986 rows × 10 columns



In [8]:

df.isnull().sum()

Out[8]:

name0
planet_status0
mass3907
mass_error_min3169
mass_error_max3169
mass_sini4784
mass_sini_error_min4947
mass_sini_error_max4947
radius1418
radius_error_min1893
dtype: int64

```
In [10]: df.fillna(df.median(numeric_only=True), inplace=True)
```

```
In [11]: df.isnull().sum()
```

```
Out[11]: name                0
planet_status              0
mass                      0
mass_error_min            0
mass_error_max            0
mass_sini                 0
mass_sini_error_min       0
mass_sini_error_max       0
radius                   0
radius_error_min          0
dtype: int64
```

```
In [12]: print("Duplicate records: ", df.duplicated())
```

```
Duplicate records: 0      False
1      False
2      False
3      False
4      False
...
5981   False
5982   False
5983   False
5984   False
5985   False
Length: 5986, dtype: bool
```

```
In [13]: df.dtypes
```

```
Out[13]: name                object
planet_status              object
mass                      float64
mass_error_min            float64
mass_error_max            float64
mass_sini                  float64
mass_sini_error_min       float64
mass_sini_error_max       float64
radius                    float64
radius_error_min          float64
dtype: object
```

```
In [14]: df["name"] = df["name"].astype(str).str.strip()
```

```
In [15]: df["planet_status"] = (
    df["planet_status"]
    .astype(str)
    .str.strip()
    .str.title()
)
print(df["planet_status"].value_counts())
```

```
planet_status
Confirmed    5986
Name: count, dtype: int64
```

```
In [16]: numeric_columns = df.select_dtypes(include="number").columns
df[numeric_columns].lt(0).sum()
```

```
Out[16]: mass                0
mass_error_min              0
mass_error_max              1
mass_sini                   0
mass_sini_error_min         0
mass_sini_error_max         0
radius                      0
radius_error_min            0
dtype: int64
```

```
In [17]: df[numeric_columns] = df[numeric_columns].mask(df[numeric_columns] < 0)
```

```
In [18]: df[numeric_columns].lt(0).sum()
```

```
Out[18]: mass                0
mass_error_min             0
mass_error_max             0
mass_sini                  0
mass_sini_error_min        0
mass_sini_error_max        0
radius                    0
radius_error_min           0
dtype: int64
```

```
In [19]: for col in numeric_columns:
        Q1 = df[col].quantile(0.25)
        Q3 = df[col].quantile(0.75)
        IQR = Q3 - Q1
        lower_limit = Q1 - 1.5 * IQR
        upper_limit = Q3 + 1.5 * IQR
        outliers = df[
            (df[col] < lower_limit) |
            (df[col] > upper_limit)
        ]
        print(col, "Outliers:", len(outliers))
```

```
mass Outliers: 2075
mass_error_min Outliers: 2811
mass_error_max Outliers: 2782
mass_sini Outliers: 1202
mass_sini_error_min Outliers: 1038
mass_sini_error_max Outliers: 1013
radius Outliers: 1058
radius_error_min Outliers: 771
```

```
In [38]: df_fill = df.fillna(method = 'bfill')
df_fill
```

C:\Users\sheriya\AppData\Local\Temp\ipykernel_19340\820761065.py:1: FutureWarning: DataFrame.fillna with 'method' is deprecated and will raise in a future version. Use obj.ffill() or obj.bfill() instead.

```
df_fill = df.fillna(method = 'bfill')
```

Out[38]:

	name	planet_status	mass	mass_error_min	mass_error_max	mass_sini	mass_sini_error_min	mass_sini_error_max	radius	radius_err
0	109 Psc b	Confirmed	5.743	0.28900	1.01100	6.38300	0.07800	0.07800	1.152	
1	112 Psc b	Confirmed	9.866	0.00500	0.00400	0.03300	0.00500	0.00400	1.110	
2	112 Psc c	Confirmed	9.866	1.78100	3.19000	11.08730	1.10000	1.10000	1.110	
3	11 UMi b	Confirmed	8.900	1.10000	1.10000	11.08730	1.10000	1.10000	1.110	
4	14 And Ab	Confirmed	8.900	0.23000	0.23000	4.68400	0.23000	0.23000	1.110	
...	...	...	...	...	...	...	...	...	...	...
5981	YBP 401 b	Confirmed	6.300	0.05000	0.05000	0.46000	0.05000	0.05000	NaN	
5982	YSES 2 b	Confirmed	6.300	0.90000	1.60000	0.00220	0.00028	0.00028	NaN	
5983	YZ Cet b	Confirmed	NaN	0.00028	0.00028	0.00220	0.00028	0.00028	NaN	
5984	YZ Cet c	Confirmed	NaN	0.00035	0.00035	0.00359	0.00035	0.00035	NaN	
5985	YZ Cet d	Confirmed	NaN	0.00038	0.00038	0.00343	0.00038	0.00038	NaN	

5986 rows × 10 columns



```
In [20]: df.replace([np.inf, -np.inf], np.nan, inplace=True)
```

```
In [21]: df.fillna(df.median(numeric_only=True), inplace=True)
```

```
In [22]: df.describe()
```

```
Out[22]:
```

	mass	mass_error_min	mass_error_max	mass_sini	mass_sini_error_min	mass_sini_error_max	radius	radius_error_min
count	5986.000000	5986.000000	5986.000000	5986.000000	5986.000000	5986.000000	5986.000000	5986.000000
mean	0.759869	0.174351	0.342393	0.982910	0.078006	0.083057	0.358280	0.039820
std	1.623854	0.938064	11.071207	1.518581	0.272364	0.400297	0.384070	0.623786
min	0.000190	0.000000	0.000000	0.000470	0.000000	0.000000	0.025430	0.000000
25%	0.370000	0.041000	0.040000	0.731500	0.049300	0.050000	0.168065	0.014000
50%	0.370000	0.041000	0.040000	0.731500	0.049300	0.050000	0.219000	0.020000
75%	0.370000	0.041000	0.040000	0.731500	0.049300	0.050000	0.281000	0.029000
max	12.934000	53.500000	850.000000	50.000000	13.000000	20.900000	6.900000	48.000000



```
In [23]: df[numeric_columns].mean()
```

```
Out[23]: mass                0.759869
mass_error_min            0.174351
mass_error_max            0.342393
mass_sini                 0.982910
mass_sini_error_min       0.078006
mass_sini_error_max       0.083057
radius                   0.358280
radius_error_min          0.039820
dtype: float64
```

```
In [24]: df[numeric_columns].median()
```

```
Out[24]: mass                0.3700
mass_error_min            0.0410
mass_error_max            0.0400
mass_sini                 0.7315
mass_sini_error_min       0.0493
mass_sini_error_max       0.0500
radius                   0.2190
radius_error_min          0.0200
dtype: float64
```

```
In [25]: df[numeric_columns].mode().iloc[0]
```

```
Out[25]: mass                0.3700
mass_error_min            0.0410
mass_error_max            0.0400
mass_sini                 0.7315
mass_sini_error_min       0.0493
mass_sini_error_max       0.0500
radius                   0.2190
radius_error_min          0.0200
Name: 0, dtype: float64
```

```
In [26]: range=df[numeric_columns].max() - df[numeric_columns].min()
print("Range: ",range)
```

```
Range: mass                12.93381
mass_error_min            53.50000
mass_error_max            850.00000
mass_sini                 49.99953
mass_sini_error_min       13.00000
mass_sini_error_max       20.90000
radius                   6.87457
radius_error_min          48.00000
dtype: float64
```

```
In [27]: df[numeric_columns].var()
```

```
Out[27]: mass                2.636902
mass_error_min            0.879964
mass_error_max           122.571633
mass_sini                 2.306088
mass_sini_error_min       0.074182
mass_sini_error_max       0.160238
radius                   0.147510
radius_error_min          0.389108
dtype: float64
```

```
In [28]: df[numeric_columns].std()
```

```
Out[28]: mass                1.623854
mass_error_min            0.938064
mass_error_max           11.071207
mass_sini                 1.518581
mass_sini_error_min       0.272364
mass_sini_error_max       0.400297
radius                   0.384070
radius_error_min          0.623786
dtype: float64
```

```
In [29]: df[numeric_columns].corr()
```

Out[29]:

	mass	mass_error_min	mass_error_max	mass_sini	mass_sini_error_min	mass_sini_error_max	radius	radius_error_min	radius_error_max
mass	1.000000	0.425470	0.101578	0.101883	0.033436	0.019474	0.204265	0.006775	0.070789
mass_error_min	0.425470	1.000000	0.040566	0.147344	0.292279	0.214043	0.030456	-0.000280	-0.000521
mass_error_max	0.101578	0.040566	1.000000	0.009087	0.023733	0.033871	-0.003047	-0.000521	-0.000280
mass_sini	0.101883	0.147344	0.009087	1.000000	0.373842	0.316914	-0.007845	-0.004696	-0.002856
mass_sini_error_min	0.033436	0.292279	0.023733	0.373842	1.000000	0.734514	-0.019119	-0.017017	-0.019119
mass_sini_error_max	0.019474	0.214043	0.033871	0.316914	0.734514	1.000000	-0.017017	-0.019119	-0.017017
radius	0.204265	0.030456	-0.003047	-0.007845	-0.019119	-0.017017	1.000000	0.070789	0.006775
radius_error_min	0.006775	-0.000280	-0.000521	-0.004696	-0.002856	-0.002226	0.070789	1.000000	0.006775
radius_error_max	0.070789	0.006775	-0.000521	-0.004696	-0.002856	-0.002226	0.006775	0.006775	1.000000

In [30]: `df[numeric_columns].skew()`

```
Out[30]: mass                4.682281
mass_error_min            33.857178
mass_error_max            75.609544
mass_sini                 10.207990
mass_sini_error_min       24.678733
mass_sini_error_max       33.613914
radius                    2.834624
radius_error_min          75.997170
dtype: float64
```

In [31]: `df.to_csv( "cleaned_analysis_ready.csv", index=False )`

```
In [32]: cleaned_df = pd.read_csv("cleaned_analysis_ready.csv")
print(cleaned_df.shape)
print(cleaned_df.head())
```

(5986, 10)

	name	planet_status	mass	mass_error_min	mass_error_max	mass_sini	\
0	109 Psc b	Confirmed	5.743	0.289	1.011	6.3830	
1	112 Psc b	Confirmed	0.370	0.005	0.004	0.0330	
2	112 Psc c	Confirmed	9.866	1.781	3.190	0.7315	
3	11 UMi b	Confirmed	0.370	1.100	1.100	11.0873	
4	14 And Ab	Confirmed	0.370	0.230	0.230	4.6840	

	mass_sini_error_min	mass_sini_error_max	radius	radius_error_min
0	0.0780	0.078	1.152	0.02
1	0.0050	0.004	0.219	0.02
2	0.0493	0.050	0.219	0.02
3	1.1000	1.100	0.219	0.02
4	0.2300	0.230	0.219	0.02

Data Cleaning and Preprocessing Documentation

1. Missing Values: Missing values were checked using `isnull().sum()`. Numerical missing values were replaced with the median because the dataset contains skewed numerical variables. Median imputation is less affected by extreme values than mean imputation.
2. Duplicate Records: Duplicate records were checked using `duplicated().sum()`.
3. Data Types: The numerical columns were converted into appropriate numeric data types. Invalid nonnumeric values were converted into NaN so that they could be handled during the missing-value treatment.
4. Categorical Consistency: The categorical columns were checked for inconsistent formatting. Extra spaces and differences in capitalization were standardized. The `planet_status` column was found to contain only the category Confirmed.
5. Invalid Values: Negative values were checked because measurements such as mass and radius should not be negative. Infinite values were also checked and replaced with missing values before performing statistical analysis.
6. Outliers: Outliers were detected using the Interquartile Range (IQR) method. A large number of outliers were found in several numerical columns. Since the dataset contains astronomical measurements, these extreme values may represent genuine observations. Therefore, the outliers were retained instead of being deleted.
7. Data Transformation: The numerical variables were examined for skewness. Log transformation can be applied to highly skewed variables such as mass, minimum mass and radius to reduce the effect of extreme values and improve the distribution.
8. Statistical Analysis: Descriptive statistics were calculated, including mean, median, minimum, maximum, range, variance and standard deviation. Correlation analysis was performed to understand relationships between numerical variables. Distribution analysis and statistical tests were also performed.
9. Final Validation: After cleaning, the final dataset was prepared for further data analysis, visualization and machine learning.

In []: